ITAS170 — Lab 6

Working with an agent II — skills, a local helper, and an agent that remembers

> **Time:** one class, Monday 9:00–2:00 · **Marks:** 20

> **Working on your own today?** This handout is written so that you can do the whole lab without help. Read the Why paragraph at the start of each part before you do its steps. Do the steps in order, and compare what you see with the You should see lines. If something goes wrong, the table at the end lists the usual causes and fixes. Keep your session log in `week06/LOG.md`, as every week.

What you will do today

Part	What you do	Time	Marks
Before you start	Check your tools; make today's local model	10 min	—
1	Write a **skill** — a saved prompt you call by name	25 min	3
2	Run the same skill in a second agent, **OpenCode**, on the local model	25 min	2
3	Have **Opus supervise** the local model on a data job — and check Opus	45 min	3
4	Install **Hermes Agent**, give it two small jobs, check both	25 min	2
5	**Make it yours**	30 min	4
Demo, next week	Explain your work to your instructor	2 min	6

In class the day runs: Intro A 9:00 · Parts 1–2 9:40 · break 10:30 · Intro B 11:00 · Parts 3–5 11:40 · finish your work 12:30–2:00. Demos are next week.

Three ideas you need today

1. An agent is a model plus a program. The model is the part that reads text and writes text: Claude Opus 5.5, or a small model on your own computer. The agent is a program wrapped around a model. It lets the model read your files, run commands and remember

things, and it decides what the model is allowed to do. Claude Code is one such program. OpenCode and Hermes Agent are two others. The same program can use different models, and the same model can sit inside different programs.

2. A local model is free and private, but weaker. A local model runs on your own computer, through a program called Ollama (week 4). Nothing you give it leaves the computer, and it costs nothing per use. But it is small — about 4 billion parameters, against a model like Opus that is many times larger (Anthropic does not publish its size) — so on a computer with no graphics card it is slow, and it makes more mistakes.

3. Check what an agent tells you. Agents sometimes say they did something they did not do, or report a number they guessed. Today you will catch this happening. The habit: after an agent says "done", look for yourself — read the file, run the command, count the rows.

Words in this handout. A **token** is a piece of a word, about four characters (week 3). A model's **context window** is how many tokens it can hold at once: its instructions, your question and its answer together. A **prompt** is the text you give a model. **Opus 5.5** is the Claude model your ITAS seat uses.

Before you start · 10 min

Where to work

The lab desktops are set up for this lab, and every step below was tested on one. **Use a lab desktop if you can.** Working from home is possible but needs more setup — read Working from home near the end of this handout first.

Step A — Check your tools

Open a terminal and run:

```
ollama --version
cd ~/code/ITAS170
git status
```

You should see `ollama version is 0.34.0` (or a higher number), and `git status` should say `On branch main` with no error. If `ollama` is older than 0.34, the `ollama launch` command in Parts 2 and 4 does not exist: see the table at the end.

Also open the **Claude** desktop app and check that you are signed in with your ITAS account and that you can see the **Code** tab.

Step B — Get the starter files

Download `itas170-week06-student.zip` from the week 6 section on Moodle and unzip it into `~/code/ITAS170/week06/`. Then:

```
cd ~/code/ITAS170/week06
ls
```

You should see `Modelfile`, `listings.csv`, `opencode.json`, `SKILL-template.md`, `supervisor-template.md`, `agents-compare-template.md`, `proposal-template.md` and `README.txt`.

Step C — Make today's local model

All three local jobs today use the same model, called `itas170-agent`. Make it once:

```
cd ~/code/ITAS170/week06
ollama pull qwen3:4b-instruct
ollama create itas170-agent -f Modelfile
ollama list
```

The download is 2.5 GB, so it can take a few minutes. If it is slow, start Part 1 while it runs and come back for `ollama create` when it finishes. If someone else used this desktop earlier today, the model may already be there — `ollama list` will show it, and you can skip ahead.

You should see `itas170-agent:latest` and `qwen3:4b-instruct` in the list (and any other models this computer already has).

Why a new model? Read the `Modelfile` (`cat Modelfile`). It has two lines that matter, and each one fixes a problem:

- `FROM qwen3:4b-instruct` — use the instruct version of the small Qwen model. The `qwen3:4b` you used in week 4 is a thinking model: before every answer it writes a long reasoning section. On a computer with no graphics card that takes minutes, even for "say hello".
- `PARAMETER num_ctx 65536` — give it a bigger context window. An agent sends thousands of tokens of its own instructions before your question arrives. On a computer with no graphics card, Ollama gives a model only 4,096 tokens unless you ask for more, and the agent's instructions do not fit. This line asks for 65,536.

Step D — Test the model

```
ollama run itas170-agent "Say hello in five words."
ollama ps
```

You should see a short greeting. The first answer can take up to a minute while the model loads; the next ones are quicker. In `ollama ps`, the `CONTEXT` column should say `65536`. If it says `4096`, you ran a different model — check the name.

Part 1 — Write a skill · 25 min · hands-only: the Output section

Why this part. Every week you write `LOG.md` entries by hand, in the same format. A **skill** is a prompt you write once, save in a file, and then call by name whenever you need it — like a function. It has a name, a one-line description, the inputs it needs, the steps to follow, and the exact shape of the output. Writing the output shape exactly is the skill's whole value: it is what makes the result the same every time. Claude Code looks for skills in `.claude/skills/NAME/SKILL.md` inside your repository.

Your skill will turn five facts about a conversation into one `LOG.md` entry. As a reminder, this is the entry format from week 1:

```
## 2026-10-19 · tool: Claude Code Desktop

**Asked:** what does git restore do to a file I have not staged?

**Got (in my words):** it throws away my change and puts back the
committed version.

**Verified by:** made a change, ran git restore, read git status.

**Next time:** ask what it does to a staged change too.
```

1. Make the folder and copy the template.

```
cd ~/code/ITAS170
mkdir -p .claude/skills/log-entry
cp week06/SKILL-template.md .claude/skills/log-entry/SKILL.md
```

The folder starts with a dot, so `ls` hides it; `ls -a` shows it.

2. Fill in the top section. Open `.claude/skills/log-entry/SKILL.md` in VS Code. Replace everything in `<angle brackets>`, brackets included. The top section, between the two `---` lines, should end up like this:

```
---
name: log-entry
description: "Draft a LOG.md entry from my summary. Use when I say log this."
disable-model-invocation: true
argument-hint: "DATE TOOL ASKED VERIFIED ANSWER"
---
```

- `name` must match the folder name, `log-entry`.
- Keep the double quotes around the description: a colon inside an unquoted description can stop the skill from loading.
- `disable-model-invocation: true` means only you can start this skill. Claude will not decide to run it by itself.
- `argument-hint` lists the five inputs, in order. Claude Code shows it as a reminder when you type the skill's name.

3. Fill in the rest. Under **Inputs**, list the five inputs with one line each: DATE (the date), TOOL (which assistant), ASKED (what you asked), VERIFIED (how you checked, or "did not check") and ANSWER (the answer you got). Under **Steps**, tell it: summarise ANSWER in one or two plain sentences and never copy it; if VERIFIED is "did not check", write exactly that; suggest one "next time" line. Delete the comment block that starts with `<!--`.

4. Write the Output section yourself — this is the hands-only part. Under **Output**, write the exact entry format shown above, with the input names where the facts go (`## DATE · tool: TOOL`, then `**Asked:** ASKED`, and so on), and end with "and nothing else". An assistant may help with steps 2–3, not this one: a format someone else wrote is not one you can hold anyone to.

5. Call it in Claude Code Desktop. Open the **Claude** app and choose the **Code** tab. Start a new session (**+ New session**). Choose **Local**, then **Select folder**, and pick `~/code/ITAS170` — the whole repository, not `week06`. In the mode selector next to the send button, choose **Manual**. Type `/` in the prompt box: `log-entry` should be in the list. Choose it, then type the five inputs after it — all on one line, with quotes around any input that has spaces. For example:

> /log-entry 2026-10-19 "Claude Code Desktop" "what does git log do?" "did not check" "It lists the commits."

Press Enter. Use a real conversation from this week for your own inputs.

You should see one entry in your format, and nothing else around it.

6. Check it, keep it, and do it again. Compare the entry with your Output section line by line. Paste it into `week06/LOG.md`. Then **rewrite the Got line in your own words** — the skill writes a draft; you decide what you understood. Call the skill a second time, with a different conversation, and do the same.

If `log-entry` is not in the `/` list, the table at the end has the fixes. Want to see a published skill? On GitHub, `mattpocock/skills` has one called `handoff` (in `skills/productivity/`) that writes a handoff file like the one in week 5. Optional.

> **Hand in:** `.claude/skills/log-entry/SKILL.md`, and two entries in `week06/LOG.md` that the skill drafted and you edited.

Part 2 — The same skill in OpenCode · 25 min

Why this part. OpenCode is an open-source agent: a program like Claude Code, which anyone can download and read. Today it runs on `itas170-agent`, on your computer, so nothing you type leaves the room. It reads the same `.claude/skills/` folder as Claude Code, so your skill works there too. Running the same skill in both shows you what changes when the model changes: same request, same program shape, very different speed — and maybe a different answer.

1. Tell OpenCode to ask first. Out of the box, OpenCode edits files and runs commands without asking you. The starter file `opencode.json` changes that. Put it where OpenCode looks for its settings:

```
mkdir -p ~/.config/opencode
cp ~/code/ITAS170/week06/opencode.json ~/.config/opencode/opencode.json
cat ~/.config/opencode/opencode.json
```

You should see `"edit": "ask"` and `"bash": "ask"`. Do this **before** you start OpenCode: it reads its settings only when it starts.

2. Start OpenCode from your repository:

```
cd ~/code/ITAS170
ollama launch opencode --model itas170-agent
```

The first time, you see `OpenCode is not installed. Install now?` with **Yes** selected. Press Enter. It installs into your home folder (no `sudo`) and opens OpenCode.

You should see the OpenCode logo, a prompt box that says Ask anything..., and under it `Build · itas170-agent:latest Ollama`. You may also see `/update to upgrade to OpenCode 2.0`

— **do not type** `/update`. This lab uses OpenCode 1; version 2 reads its settings differently, and your "ask first" rule would stop working.

3. Call your skill. Type `/skills` and press Enter. A list opens; choose `log-entry` with the arrow keys and press Enter. You should see `/log-entry` in the prompt box. Type the same five inputs you used in Part 1 after it, and press Enter.

4. Wait — and time it. On a lab desktop the first answer takes about two minutes. The model first reads OpenCode's own instructions (about 7,000 tokens) and then yours. Do not press any keys while it works; the moving dots at the bottom mean it is busy. When it finishes, OpenCode shows the time under the answer, for example `Build · itas170-agent:latest · 52.2s`.

You should see an entry in your format. Check it line by line against your Output section. Did the small model keep the shape?

5. If it asks permission. If OpenCode shows **Permission required** with **Allow once**, **Allow always** and **Reject**, read what it wants to do. You asked for text, not a file, so the answer is usually **Reject**: use the arrow keys to choose it, then press Enter. Write down what it tried.

6. Quit by typing `/exit` and pressing Enter.

Copy `agents-compare-template.md` to `agents.md` and fill in the Claude Code Desktop and OpenCode columns from what you just saw:

```
cp ~/code/ITAS170/week06/agents-compare-template.md \
    ~/code/ITAS170/week06/agents.md
```

> **Hand in:** the OpenCode output as its own `LOG.md` entry (note the time it took); the first two columns of `week06/agents.md`.

Part 3 — Opus supervises the local model · 45 min · hands-only: three checks

Why this part. A small local model is free, fast to call, and keeps data on your computer, but it makes mistakes. Opus is far more careful, but every use costs money and sends your text to Anthropic. A common way to use both at work: the **local model does the bulk of the work**, and **a large model plans the job and checks the results**. The large model is the supervisor. And **you check the supervisor** — because it can be wrong too.

The job is a shopping search. `listings.csv` has 24 laptop listings, made up for this lab in the style of an online marketplace (open it and read a few). The question:

Which listings are a working 16-inch MacBook Pro with 64 GB of RAM or more?

Read the titles carefully: some listings are designed to trick a quick reader. Storage (SSD) is not RAM. A box, or a laptop sold for parts, is not a working laptop.

Work in **Claude Code Desktop**, in **Manual** mode, in `~/code/ITAS170` (the same as Part 1, step 5). Give it the four requests below **one at a time**, typed or pasted exactly. Wait for each to finish before the next.

Request 1 — the first pass.

> Read week06/listings.csv. For each listing, run the local model on its title with `ollama run itas170-agent`, using exactly this prompt followed by the title: "Read this laptop listing. Answer with three things separated by commas: the screen size in inches, the RAM in GB, and yes or no: is it a complete, working laptop?" Save the answers to week06/pass1.csv with the listing id. Do not judge the listings yourself yet.

Claude will show you what it wants to do and ask permission. It may first ask to create a small script file, then ask to run it. **Read the command (or the script) before you allow it.** Find three things in it: where it reads each listing, where it calls `ollama run`, and where it writes `pass1.csv`. Then allow it — once; do not choose an option that allows it from now on. The 24 calls take under a minute on a lab desktop.

You should see a new file, `week06/pass1.csv`, with 24 answers. Open it.

Request 2 — the check.

> Compare every answer in week06/pass1.csv with its listing title. Count the rows that are fully right, and list the wrong ones with the reason.

You should see a count out of 24 and a list of wrong rows. The local model usually gets several wrong.

Request 3 — a better prompt.

> Write a better prompt for the local model that fixes those mistakes. Run it on all 24 listings again and save the answers to week06/pass2.csv. Check them the same way and give me both counts.

Read the new prompt Claude writes before you allow the command. What did it add?

Request 4 — the answer.

> Using pass2.csv and your own checks, which listings are a working 16-inch MacBook Pro with 64 GB of RAM or more? Give the ids and one reason for each.

Now check Opus yourself (hands-only, 10 min). Copy the write-up template:

```
cp ~/code/ITAS170/week06/supervisor-template.md \
    ~/code/ITAS170/week06/supervisor.md
```

Pick three rows: one that Opus said the local model got **wrong**, one it said was **right**, and one in its **final answer**. Read each title in `listings.csv` yourself. Do you agree? Fill in every section of `supervisor.md`: the two counts, the final answer, your three checks, the `ollama run` command and what each part of it does, and two sentences on who did what and why.

Add two rows to `scoreboard.csv` (the file at the root of your repository, from week 4). Use your own counts and times; these show the shape:

```
6,find the laptops,itas170-agent (pass 1),partly,read 3 titles,20,19 of 24
6,find the laptops,itas170-agent (pass 2),partly,read 3 titles,27,23 of 24
```

`correct` (the fourth column) is `yes` if all 24 rows were right, `partly` if most were, `no` if fewer than half. Put the count in the last column. Avoid commas inside a column — they start a new one.

> **Hand in:** `pass1.csv`, `pass2.csv`, `supervisor.md`, two scoreboard rows.

Part 4 — Hermes Agent: an agent that remembers · 25 min

Why this part — and why Hermes, if Opus is smarter? You are right that Opus 5.5 is far more capable than the small model Hermes will use today. But Hermes is not a model: it is a program, like Claude Code (see Three ideas, idea 1). Today it runs on the small local model, so it is slower and makes more mistakes than Claude Code with Opus. That is the model, not the program.

Hermes is here because it is a different **kind** of agent:

- **It stays on.** Claude Code is a session you open, use and close. Hermes is built to run all the time on a computer or server, wait for messages from Telegram, Discord or email, and run jobs on a schedule — with nobody watching.
- **It can use any model**, including a free local one. For a job that never stops, that matters: an agent that wakes up every hour and sends thousands of tokens each time costs real money on a paid model (week 3), and nothing on a local one. Your data also stays on your computer.
- **It is open source.** Anyone can read exactly what it does. That is how we found that, out of the box, it borrows other programs' saved logins — including your Claude Code login (step 2 below).

> **A short history: OpenClaw and the $200 plan.** Hermes is not the first agent of this kind. In late 2025 a developer, Peter Steinberger, built a personal assistant you could message from WhatsApp or Telegram. It ran on your own computer, used `SKILL.md` skills like the one you wrote in Part 1, and woke itself up every 30 minutes (a heartbeat) to check whether there was anything to do. He named it Clawdbot, after Claude. After Anthropic objected to the name it became Moltbot, then **OpenClaw** (January 2026). By the end of that month it had passed 100,000 stars on GitHub.
>
> Many users connected it to a **Claude Max subscription** — $100 or $200 a month, flat rate, meant for a person at a keyboard — instead of an API key that charges per token. A program that never sleeps uses far more than a person does. Anthropic said in July 2025 that one user had used "tens of thousands" of dollars of model time on a $200 plan (running Claude Code non-stop), and an industry analysis in October 2026 estimated that a fully used $200 plan is worth up to about $11,700 a month at API prices. Anthropic's terms already said a subscription login is for its own apps. It closed the gap in steps: in January 2026 it blocked tools that pretended to be Claude Code, and from **4 April 2026** tools like OpenClaw stopped drawing on subscription limits — users paid extra, or used an API key. In October 2026 it added a separate monthly allowance of API credits to its Max and Team plans.
>
> **What it would cost.** OpenClaw's documentation puts one heartbeat with a full conversation at about 100,000 tokens. Every 30 minutes, that is 4.8 million tokens a day — before the agent does any real work. At Opus 5.5's API price, that is between about $1 a day (if the conversation is cached, week 4) and about $24 a day (if it is not): up to roughly $700 a month for an agent that mostly waits. On a local model: nothing but electricity.
>
> OpenClaw also became a security story: a bug that let an attacker run commands on the user's computer after a single click, 341 harmful skills found in its shared skills store, and tens of thousands of copies left reachable from the internet. That is what an always-on agent with your logins looks like when it goes wrong — and why the first thing you do with Hermes, below, is stop it borrowing yours. (Sources: Anthropic; TechCrunch, 27 Jan and 4 Apr 2026; The Register, 20 Feb and 6 Apr 2026; The Hacker News, Feb 2026; SemiAnalysis, Oct 2026; docs.openclaw.ai.)

An agent that runs while nobody watches is where this course's safety thread leads: week 5's leash, week 8's sandbox, week 9's server. Today you install one, switch off the risky default, give it two small jobs, and **check both** — because a small model can tell you it did something it did not do.

1. Install Hermes and connect it to the local model:

```
cd ~/code/ITAS170
ollama launch hermes --config --model itas170-agent
```

At `Hermes is not installed. Install now?`, press Enter (**Yes** is selected). It downloads its own copy of Python and some tools, so it takes a few minutes and about 3 GB in your home folder. No `sudo`.

You should see `Hermes Agent install complete` near the end, and a line telling you to reload your shell. **Open a new terminal** (so the `hermes` command is found), and go back to your repository:

```
cd ~/code/ITAS170
```

2. Switch off the risky default — before anything else:

```
hermes config set auth.adopt_external_logins false
```

You should see `✓ Set auth.adopt_external_logins = False`. Out of the box, Hermes looks for logins that other programs saved on your computer, including Claude Code's, and uses them. That can sign you out of Claude Code, and in this course your ITAS seat is used only in Claude's own apps (see the OpenClaw story above). This one setting tells Hermes to leave other programs' logins alone.

3. Job 1 — one tool.

```
hermes chat --oneshot -t terminal \
    -q "How many lines are in week01/access.log? Use a terminal command."
```

`--oneshot` means: answer this one question, then stop. `-t terminal` gives Hermes one kind of tool, the terminal. Without `-t`, Hermes loads more than fifteen kinds of tool (web, browser, images and more), and the small model spends minutes reading their instructions and then often gets the answer wrong. Fewer tools is also less it can do: week 5's least privilege.

Expect about a minute on a lab desktop. You should see a line with a `$` and `wc -l week01/access.log` — that is the command Hermes ran — then its answer, then `Duration:`. Now check it yourself:

```
wc -l week01/access.log
```

Do the two numbers match?

4. Job 2 — memory, checked. Ask Hermes to remember something. Then look in the file where its memory lives:

```
hermes chat --oneshot -t memory \
    -q "Remember this: my ITAS 170 demo is on Monday 26 October."
cat ~/.hermes/memories/MEMORY.md
```

What did Hermes say? Is the note in the file? (If `cat` says No such file or directory, the file does not exist: nothing was saved.) With this vague wording, the small model saves the note only some of the time — but it always says it remembered. Now ask more exactly:

```
hermes chat --oneshot -t memory \
    -q "Save this to your memory: my ITAS 170 demo is on Monday 26 October."
cat ~/.hermes/memories/MEMORY.md
```

When Hermes really uses its memory tool, a line with the word `memory` appears in its output under Initializing agent. Last, ask in a new session — nothing carries over except the memory file:

```
hermes chat --oneshot -t memory -q "When is my ITAS 170 demo?"
```

5. Write it down. Fill in the Hermes column of `agents.md`, and the two sentences at the bottom. One `LOG.md` entry for each job: what you asked, what Hermes said, and what you found when you checked.

> **Hand in:** `week06/agents.md` complete; a `LOG.md` entry for each Hermes job, with your check beside it.

Part 5 — Make it yours · 30 min

Three routes. Each is worth the full four marks. Pick one.

Route A — a skill of your own. Pick a task you repeat: a weekly summary, a checklist, a message you often send. Make `.claude/skills/NAME/SKILL.md` from the template, with at least one input. Call it in Claude Code Desktop and in OpenCode (restart OpenCode first, so it sees the new skill). Did both keep the output shape? Write your answer in `week06/route-a.md`.

Route B — supervise a job of your own. 15 to 30 rows of your own data — made up, or public; never private — and one question about them. Have Opus run the local model over the rows, check the answers and improve the prompt, as in Part 3. Report both counts and three rows you checked by hand in `week06/route-b.md`. (Or use the listings with a different local model — ask first if it is over 5 GB — and compare its pass 2 with `itas170-agent`.)

Route C — one page for someone who will not read a terminal. Pick a workflow you know: a job you have had, a club, a team, another course. Copy `proposal-template.md` to `proposal.md` and write one page for the person who runs it: how the work is done today, in five lines; where an agent would help; exactly what it may read and what it may change; the first thing that would go wrong, and who would notice; **three measures that would show, after a month, whether it was worth it**; and the cost, from week 3's table. No code. End with one sentence: do it, do it smaller, or do not do it.

Then commit and push everything:

```
cd ~/code/ITAS170
git add -A
git status
git commit -m "Week 6: a skill, two more agents, a supervised local model"
git push
```

Read the `git status` list before you commit: you should see your `week06/` files and `.claude/skills/`. If you see a file you do not want in git, stop and ask.

> **Hand in:** the route's files, in the repo, pushed.

Marking — 20 marks

	Component	Marks
C1	A skill — `log-entry` with front matter, inputs, steps and an exact output; called twice; the Got lines yours	3
C2	The same skill in OpenCode — on the local model, timed, the output checked against the format	2
C3	Opus supervises the local model — two passes, both counts, the final list, three checks of your own, two scoreboard rows	3
C4	Hermes — installed safely, two jobs, each claim checked; `agents.md` complete	2
C5	**Make it yours** — your own skill, a supervised job of your own, or a one-page proposal	4
DU	**Demonstrates understanding** — marked in person	**6**
		20

The **make it yours** criterion is worth **4 marks** — more than any single required part. `DU` is awarded from your show+tell demo next week, not from your files. Be ready for: "Show me the `ollama run` command Opus ran in Part 3, and tell me what each part of it does." Then: "Which row did the local model get wrong, and how did you check Opus?" and "Why did we give Hermes only one kind of tool?"

Submitting

Your repository is the submission — everything pushed by the end of Part 5. For the record, also zip `~/code/ITAS170/week06/` and upload it to the week 6 section on Moodle by **Sunday 25 October**. Please commit and push your work to demo next week.

On using AI for this lab

Agents do much of the typing today — that is the point. Three things are **yours alone**: you write your skill's Output section; you check three of Opus's verdicts by reading the titles yourself; and you check what Hermes tells you with a command of your own (`wc -l`, `cat`). Every session goes in `LOG.md`. The safety rules still hold: switch off Hermes's login borrowing before its first job; never give an agent a password or a key; and the bypass permission mode stays off until week 8.

Working from home

Everything in this handout was tested on a lab desktop (Ubuntu). At home it can work, but check these first.

What your computer needs

Memory (RAM)	**16 GB or more.** Today's model with its big window takes about 12 GB, so close other programs. With 8 GB, do Parts 1 and 3 at home (see below) and Parts 2 and 4 on a lab desktop. Check: Windows, Settings → System → About; Mac, Apple menu → About This Mac; Linux, `free -h`.
Claude desktop app	Download it from `claude.ai/download` and sign in with your **ITAS** Claude account. You should see the **Code** tab.
Ollama	From `ollama.com/download`. Check with `ollama --version`: you need 0.34 or later. The Mac and Windows apps update themselves; restart Ollama if it offers an update.
Your repository	If this computer already has `~/code/ITAS170`, run `git pull` in it. If not, clone it from GitHub as in week 2 (that needs your SSH key on this computer).
Disk space	About 8 GB free: the model (2.5 GB), Hermes (3 GB) and OpenCode.

With 8 GB of memory: in Part 3, use `qwen3:4b-instruct` instead of `itas170-agent` in request 1 — the listing prompts are short and fit in its normal window. Do Parts 2 and 4, which need the big window, on a lab desktop.

Mac (Apple silicon) or Linux: the commands work as written. On a Mac the model runs on the graphics chip, so it is much faster than on a lab desktop. If a command says `git` is missing on a Mac, run `xcode-select --install`.

Windows 11: this was **not tested** on Windows; use a lab desktop if you can. If you must use Windows, do the terminal steps in **PowerShell** and note these differences:

In this handout	On Windows (PowerShell)
a long command split with `\` at the end of a line	type it all on one line, without the `\`

In this handout	On Windows (PowerShell)
`mkdir -p ~/.config/opencode`	`mkdir -Force ~/.config/opencode`
`wc -l week01/access.log`	`(Get-Content week01/access.log).Count`
`cat ~/.hermes/memories/MEMORY.md`	`cat $env:LOCALAPPDATA/hermes/memories/MEMORY.md`
`source ~/.bashrc`, or "open a new terminal"	close PowerShell and open a new one
`watch -n 2 ollama ps`	run `ollama ps` again when you want to look
`ollama launch opencode` stops: Install the following first: npm	install Node.js from `nodejs.org`, open a new PowerShell, and run it again

`cd`, `ls`, `cp`, `cat`, `git`, `ollama` and `hermes` work as written in PowerShell. In Part 3, Claude Code Desktop runs its commands in PowerShell on Windows, so its commands will look different from a classmate's on a lab desktop. That is fine — read them the same way: where it reads each listing, where it calls `ollama run`, where it writes the file. After installing Ollama, Node.js or Git, restart the Claude app so it can find them.

If something goes wrong

Before you start

What you see	What it means	What to do
`ollama: command not found`	Ollama is not installed on this computer.	On a lab desktop, tell your instructor. At home: Working from home.
`ollama --version` is lower than 0.34, or `unknown command "launch"`	This Ollama is too old for `ollama launch`.	Update Ollama (Working from home), or use a lab desktop.
`could not connect to ollama`	The Ollama service is not running.	Lab desktop: tell your instructor. At home: start the Ollama app.
`ollama pull` is very slow	Everyone is downloading 2.5 GB at once.	Start Part 1 while it runs; the second person on a desktop can skip the pull.
`ollama create` says the model was not found	The pull has not finished, or failed.	`ollama pull qwen3:4b-instruct` again, then `ollama create`.
`ollama ps` shows `CONTEXT 4096`	You ran `qwen3:4b-instruct`, not `itas170-agent`.	Use `itas170-agent`; if `ollama list` does not show it, redo Step C.
The first answer from the model takes a long time	It is loading 12 GB into memory, on the processor.	Wait about a minute; the next answers are quicker.

Part 1

What you see	What it means	What to do
`log-entry` is not in the `/` list	Claude Code did not find the file, or the session started before you made it.	`ls ~/code/ITAS170/.claude/skills/log-entry/SKILL.md`; check the Code tab's folder is `~/code/ITAS170` (not `week06`); start a new session (**+ New session**).
`ls` shows no `.claude` folder	Names starting with a dot are hidden.	`ls -a`.
The skill runs but ignores your format	The Output section is not exact, or the top section is broken.	Check the three `---` lines, the quotes around the description, and that `name` is `log-entry`.
The output has extra chatter around the entry	The Output section is not strict enough.	Add "Output only the entry — no introduction, no closing line." That edit is the lesson.

Part 2

What you see	What it means	What to do
`OpenCode is not installed. Install now?` does not appear, and nothing opens	The install failed (often the network).	Run `ollama launch opencode --model itas170-agent` again.
`/log-entry` is not offered when you type `/log`	OpenCode lists skills under `/skills`, not in the `/` list.	Type `/skills`, Enter, choose `log-entry`. (Typing `/log-entry` and your inputs in full also works.)
`log-entry` is not in `/skills`	OpenCode was started before the skill existed, or from another folder.	`/exit`, `cd ~/code/ITAS170`, start it again.
The answer takes minutes	Normal on a computer with no graphics card: it reads about 7,000 tokens of its own instructions first.	Wait. In a second terminal, `watch -n 2 ollama ps` shows the model working (100% CPU). Ctrl+C stops `watch`.
OpenCode edits a file without asking	Its settings file is missing, so it uses its default: allow everything.	`cat ~/.config/opencode/opencode.json`; copy it again (step 1); `/exit` and start OpenCode again.
You typed `/update` and OpenCode changed	You are now on OpenCode 2.	Ask your instructor; note it in your log. Parts 3–5 do not need OpenCode.
OpenCode says the request was too large, or that it compacted the conversation	The model tried to read a big file and ran out of window.	Ask about something small; your skill needs no files.

Part 3

What you see	What it means	What to do
Opus answers from the titles without running the local model	It can, and it is faster — but that is not the job.	Reply: "Run the local model first with `ollama run itas170-agent`, as I asked. Do not judge the listings yet."
`pass1.csv` has only one answer	`ollama run` read the rest of the list as its own input, and the loop stopped.	Tell Claude what happened. It should add `< /dev/null` to the `ollama run` command. Read the fix before you allow it. Worth a log line.
Claude asks permission again and again	Manual mode asks before each command and each new file. That is the point.	Read each one, then allow it once. Avoid any option that allows it from now on.
A row says a 17-inch MacBook Pro	The model read the listing's id number as part of the screen size.	A good catch for your three checks. Ask Claude to send the title only.

Part 4

What you see	What it means	What to do
`hermes: command not found`	This terminal started before Hermes added itself to your `PATH`.	Open a new terminal, or run `source ~/.bashrc`.
The install stops with an error about `libatomic.so.1`	A system library is missing.	Lab desktop: tell your instructor. At home on Linux: `sudo apt install libatomic1`, then run the install again.
exceeds the available context size (4096 tokens)	Hermes is using a model without the bigger window.	`ollama launch hermes --config --model itas170-agent` again.
Hermes takes more than three minutes	You left out `-t`, so it loaded every kind of tool.	Ctrl+C, then run it again with `-t terminal` or `-t memory`.
Hermes says it saved something, but `MEMORY.md` does not exist	It told you it did something it did not do.	That is Job 2's point. Write it down, then use the exact wording: "Save this to your memory: …".
The first "Remember this" **did** save the note	The small model is not consistent: in testing it saved 3 times out of 10, and said it had every time.	Fine — write down what happened. Then run the second wording anyway, and read the file again.
Claude Code asks you to sign in again	Hermes used your Claude login before you switched off step 2's setting.	Sign in again in Claude; run step 2's command; note it in your log.

Pushing

What you see	What it means	What to do
`git push` rejected	Your GitHub copy has a commit this computer does not.	`git pull`, then `git push`.
`git status` lists `.env` or a password file	It should be in `.gitignore` (week 2).	Do not commit it. `git restore --staged FILE` if you added it.

The habit this lab adds: **check the agent's claim, not its tone.** A skill is a prompt you can hold to a format. A local model does the bulk work and makes mistakes. A supervisor model catches most of them — and you check the supervisor.